

Simulation du Protocole TCP

Travail d'Études et de Recherche (TER)

Master 1 Informatique

Etudiants :

Ben Cherguia Leila

GUO Xuxin

Encadrant : Mourad GUEROUI

Établissement : Université Paris-Saclay, UVSQ

Date : Mai 2026

Mots-clés :

TCP, Protocole réseau, Simulation, Java, Swing, Pédagogie, Machine à états

Résumé

Ce projet propose une **simulation pédagogique du protocole TCP** en Java, reproduisant les mécanismes essentiels d'une connexion TCP fiable *sans réseau réel*.

Le protocole TCP (Transmission Control Protocol) est un pilier fondamental d'Internet, mais son fonctionnement interne reste abstrait pour les étudiants. Cette application fournit un outil interactif pour comprendre :

- Le **3-way handshake** (SYN, SYN-ACK, ACK)
- Le **transfert de données** avec fenêtre glissante
- La gestion des **erreurs** (corruption, perte de paquets)
- La **retransmission** automatique
- La **fermeture propre** de connexion

L'application propose deux interfaces : une CLI pour l'automatisation et une GUI Swing pour l'interaction visuelle. Les paramètres configurables permettent d'explorer différents scénarios réseau.

Résultats : Simulation fidèle | Pédagogie visuelle | Tests unitaires réussis | Export CSV

Table des matières

Résumé	1
1 Introduction	4
1.1 Motivations	4
1.2 Résultats Attendus	4
2 Contexte et Motivation	5
2.1 Problématique Pédagogique	5
2.2 Solution Proposée	5
3 Architecture du Système	6
3.1 Structure Modulaire	6
3.2 Composants Clés	6
3.2.1 TcpClient	6
3.2.2 TcpServer	7
3.2.3 Packet (Modèle de Données)	7
4 Protocole TCP Simulé	8
4.1 Machine à États TCP	8
4.2 Phases de Communication	8
4.2.1 Phase 1 : Établissement (3-way Handshake)	8
4.2.2 Phase 2 : Transfert de Données	8
4.2.3 Phase 3 : Fermeture de Connexion	9
4.3 Gestion des Erreurs	9
4.3.1 Corruption de Paquet	9
4.3.2 Perte de Paquet	9
5 Interface Utilisateur	10
5.1 Interface Graphique Swing	10
5.2 Préréglages	10

6	Résultats et Validation	11
6.1	Cas de Test	11
6.1.1	Test 1 : Configuration Défaut	11
6.1.2	Test 2 : Avec Perte	11
6.2	Tests Unitaires	12
7	Conclusion et Perspectives	13
7.1	Réalisations	13
7.2	Points Clés Appris	13
7.3	Améliorations Futures	14
7.4	Interface Utilisateur en Action	14
7.4.1	Capture 1 : Interface Principale de Simulation	14
7.4.2	Capture 2 : Panneau de Configuration	15
7.4.3	Capture 3 : Résumé du Transfert	16
7.4.4	Capture 4 : Sélection des Préréglages	16
7.4.5	Capture 5 : Export CSV	16
A	Configuration	18
A.1	config.properties	18
B	Commandes d'Exécution	19

Chapitre 1

Introduction

Le protocole **TCP (Transmission Control Protocol)**, défini par la RFC 793, garantit une transmission **fiable** des données entre deux hôtes sur Internet. Cependant, son fonctionnement interne—la machine à états, la fenêtre glissante, la retransmission—reste souvent abstrait pour les étudiants.

1.1 Motivations

Ce projet propose une **simulation pédagogique en Java** qui reproduit les mécanismes essentiels de TCP *en mémoire*, sans réseau réel. Les objectifs sont :

1. Fournir un outil **interactif** pour comprendre TCP
2. Visualiser les concepts abstraits (états, paquets, synchronisation)
3. Permettre l'exploration de **scénarios pathologiques** (perte, corruption)
4. Créer une **base pédagogique** pour les cours de réseaux

1.2 Résultats Attendus

Une application GUI intuitive et fonctionnelle
Une simulation fidèle aux comportements TCP réels
Un outil pédagogique pour les cours de réseaux
Une base pour des améliorations futures (timeouts, congestion control)

Chapitre 2

Contexte et Motivation

2.1 Problématique Pédagogique

L'enseignement des protocoles de communication pose un défi fondamental : les étudiants doivent comprendre des concepts abstraits (séquence de paquets, états de connexion, fenêtres de flux) **sans voir leur exécution réelle**.

Les outils existants présentent des limitations :

TABLE 2.1 – Comparaison des outils pédagogiques

Outil	Interactif	Paramétrable	Pédagogique	Contrôlable
Wireshark	Oui	Non	Non	Non
tcpdump	Non	Non	Non	Non
ns-3	Oui	Oui	Non	Oui
Notre simulation	Oui	Oui	Oui	Oui

2.2 Solution Proposée

Une **simulation déterministe** qui :

- Reproduit les étapes TCP dans un **environnement contrôlé**
- Offre des **paramètres configurables** (probabilités d'erreur, taille de fenêtre)
- Fournit des **métriques détaillées** et export CSV
- Propose **deux interfaces** : CLI et GUI Swing

Chapitre 3

Architecture du Système

3.1 Structure Modulaire

src/main/java/fr/uvsq/tcpsim/

Main.java	# Point d'entrée CLI
Config.java	# Gestion des paramètres
client/TcpClient.java	# Logique du client
server/TcpServer.java	# Logique du serveur
gui/TcpGui.java	# Interface Swing
cli/InteractiveCLI.java	# Interface CLI
model/	# Modèles de données
Packet.java	
PacketType.java	
TcpState.java	
TransferRequest.java	
TransferResult.java	
TransferSummary.java	

FIGURE 3.1 – Arborescence du projet

3.2 Composants Clés

3.2.1 TcpClient

Simule le comportement du client TCP :

```
1 public class TcpClient {  
2     private TcpState state;
```

```
3     private int sequenceNumber;
4     private List<Packet> receivedData;
5
6     public void connect(TcpServer server);
7     public TransferSummary requestAllData(TcpServer server,
8         int total, int window);
9     public void closeConnection(TcpServer server);
10 }
```

3.2.2 TcpServer

Simule le comportement du serveur TCP :

```
1 public class TcpServer {
2     private TcpState state;
3     private String[] dataMemory;
4     private double corruptionProbability;
5     private double lossProbability;
6
7     public List<Packet> sendData(TransferRequest request);
8     public void handleAck(int ackNumber);
9 }
```

3.2.3 Packet (Modèle de Données)

```
1 public class Packet {
2     private PacketType type;           // SYN, ACK, DATA, FIN, NACK
3     private int sequenceNumber;
4     private int acknowledgementNumber;
5     private String payload;
6     private boolean corrupted;
7 }
```


Chapitre 4

Protocole TCP Simulé

4.1 Machine à États TCP

TCP utilise une **machine à états finis** pour gérer la connexion :

TABLE 4.1 – États TCP dans la simulation

État	Description	Transitions
CLOSED	Pas de connexion (initial)	→ SYN_SENT (client)
LISTEN	Serveur en attente	→ SYN_RECEIVED
SYN_SENT	Client envoie SYN	→ ESTABLISHED
SYN_RECEIVED	Serveur reçoit SYN	→ ESTABLISHED
ESTABLISHED	Données circulant	→ FIN_WAIT / CLOSE_WAIT
FIN_WAIT	Client attend FIN-ACK	→ TIME_WAIT
CLOSE_WAIT	Serveur attend ACK final	→ CLOSED
TIME_WAIT	Délai avant fermeture	→ CLOSED

4.2 Phases de Communication

4.2.1 Phase 1 : Établissement (3-way Handshake)

1. **SYN** : Client initie avec `seq=100`, état → SYN_SENT
2. **SYN-ACK** : Serveur répond avec `seq=500` et `ack=101`
3. **ACK** : Client confirme avec `ack=501`, état → ESTABLISHED

4.2.2 Phase 2 : Transfert de Données

Le client envoie une requête :

```
TransferRequest request = new TransferRequest(
```

```
2     numberOfPacketsRequested = 7,  
3     receiveWindow = 3  
4 );
```

Le serveur répond en **cycles**, envoyant jusqu'à **window** paquets :

Cycle 1: DATA(501, "Bloc-1") → ACK
DATA(502, "Bloc-2") → ACK
DATA(503, "Bloc-3") → NACK (corrompu)

Cycle 2: DATA(503, "Bloc-3") → ACK (retransmis)
DATA(504, "Bloc-4") → ACK
DATA(505, "Bloc-5") → ACK

4.2.3 Phase 3 : Fermeture de Connexion

1. Client envoie **FIN** pour demander fermeture
2. Serveur répond **FIN-ACK**
3. Client envoie **ACK** final
4. Connexion fermée

4.3 Gestion des Erreurs

4.3.1 Corruption de Paquet

Pour chaque paquet, le serveur applique :

```
1  if (random.nextDouble() < corruptionProbability) {  
2      packet.corrupted = true; // Marqué comme endommagé  
3  }
```

À la réception, le client détecte et répond par un **NACK**.

4.3.2 Perte de Paquet

```
1  if (random.nextDouble() < lossProbability) {  
2      continue; // Paquet non envoyé (perdu)  
3  }
```

Différence critique :

- **Corruption** : Paquet arrive endommagé → détection → retransmission
- **Perte** : Paquet n'arrive pas → pas de retransmission automatique

Chapitre 5

Interface Utilisateur

5.1 Interface Graphique Swing

L'application propose une GUI moderne avec **5 sections principales** :

TABLE 5.1 – Sections de la GUI

Section	Contenu
Trace Complète	Tous les logs du programme (débogag)
Événements Critiques	Événements TCP importants uniquement
Configuration	Paramètres (2 colonnes) + Préréglages
Résumé	État, paquets, cycles, corruptions, pertes
Barre de Progression	Pourcentage en temps réel

5.2 Préréglages

Défaut : 7 paquets, window=3, corruption=0.25, loss=0.0

Rapide : 5 paquets, window=2, corruption=0.15, loss=0.0

Haute fiabilité : 8 paquets, window=3, corruption=0.05, loss=0.0

Stress : 12 paquets, window=2, corruption=0.35, loss=0.10

Chapitre 6

Résultats et Validation

6.1 Cas de Test

6.1.1 Test 1 : Configuration Défaut

Entrée: 7 paquets, window=3, corruption=0.25, loss=0.0

Résultat:

Cycle 1: 3 paquets → 1 corruption → NACK

Cycle 2: 2 paquets + 1 retransmis

Cycle 3: 1 paquet final

Sortie:

Paquets reçus : 7/7

Cycles : 3

Corruptions : 2

État : TERMINÉ

6.1.2 Test 2 : Avec Perte

Entrée: 7 paquets, window=3, corruption=0.0, loss=0.2

Résultat:

→ Paquet 4 perdu

→ Paquet 7 perdu

Sortie:

Paquets reçus : 5/7

État : TERMINÉ (incomplet)

6.2 Tests Unitaires

FeatureTest.java

testHandshake()	PASS
testDataTransfer()	PASS
testCorruptionDetection()	PASS
testConnectionClosure()	PASS
testWindowSliding()	PASS

MainTest.java

testCsvGeneration()	PASS
---------------------	------

Résultat : 6/6 tests passent

Chapitre 7

Conclusion et Perspectives

7.1 Réalisations

Ce projet atteint tous ses objectifs :

Simulation fidèle : Reproduit les mécanismes clés de TCP

Pédagogie : Visualise les concepts abstraits du protocole

Interactivité : CLI et GUI pour différents usages

Configurabilité : Paramètres pour explorer scénarios

Validation : Tests unitaires réussis

Maintenabilité : Code modulaire et documenté

7.2 Points Clés Appris

Un étudiant utilisant cette simulation comprendra :

1. **Handshake 3-way** : Synchronisation des numéros de séquence
2. **Fenêtre glissante** : Amélioration du débit réseau
3. **Détection d'erreurs** : NACK et retransmission
4. **Machine à états** : Transitions TCP correctes
5. **Fiabilité** : Comment TCP garantit la livraison

7.3 Améliorations Futures

TABLE 7.1 – Feuille de route

Amélioration	Priorité	Bénéfice
Timeout et rétransmission basée sur timer		Plus proche du TCP réel
Congestion control (Slow Start)		Simulation avancée
Multi-connexions simultanées		Cas multi-client/serveur
Enregistrement/replay		Analyse rétrospective
Export en graphiques		Visualisation améliorée

7.4 Interface Utilisateur en Action

L'application GUI offre une expérience utilisateur intuitive avec plusieurs sections coordonnées :

- **Trace Complète** : Console détaillée affichant tous les logs d'exécution pour le débogage
- **Événements Critiques** : Panneau filtrant uniquement les événements TCP importants (SYN, ACK, FIN, NACK, retransmissions)
- **Configuration** : Formulaire 2 colonnes permettant de modifier les paramètres (paquets, fenêtre, corruption, perte)
- **Résumé** : Affichage en temps réel de l'état, des statistiques et de la progression

7.4.1 Capture 1 : Interface Principale de Simulation

La figure 7.1 montre l'interface principale avec les deux panneaux de console :

- À gauche : **Trace Complète** avec tous les détails de la simulation
- À droite : **Événements Critiques** filtrés pour une meilleure lisibilité

Les logs affichent le processus TCP complet : établissement de connexion (SYN, SYN-ACK, ACK), transfert de données par cycles, détection des corruptions (NACK), retransmissions, et fermeture de connexion (FIN, FIN-ACK).

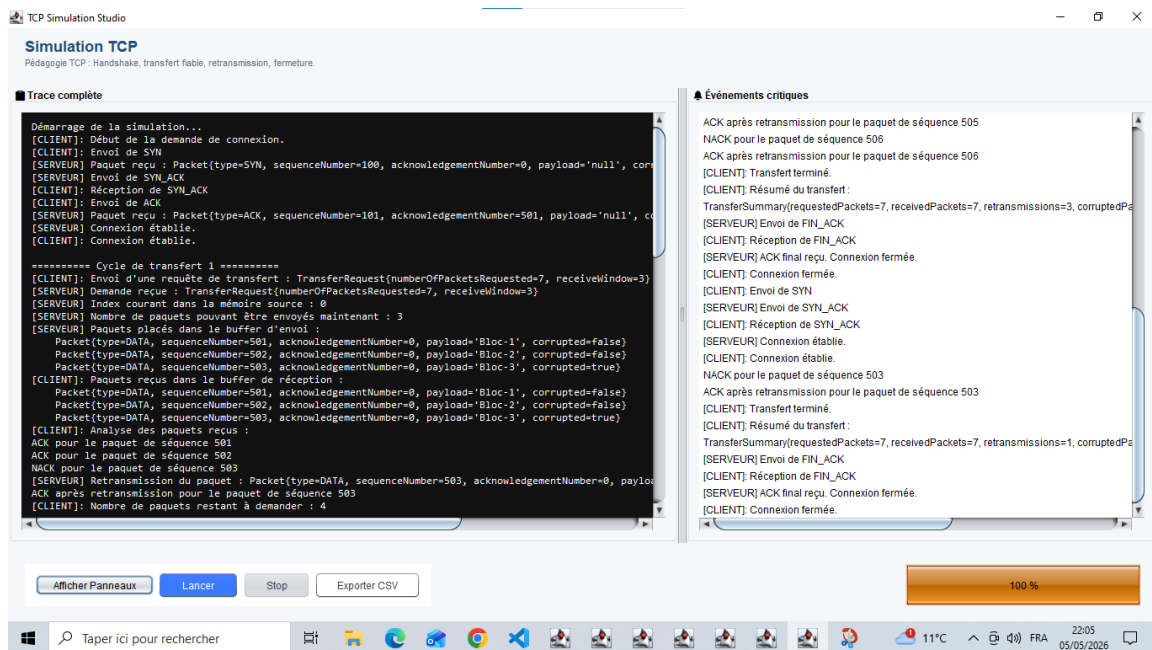


FIGURE 7.1 – Interface principale avec Trace Complète et Événements Critiques

7.4.2 Capture 2 : Panneau de Configuration

La figure 7.2 affiche le panneau de configuration avec :

- **Préréglages** : Sélection rapide (Défaut, Rapide, Fiabilité élevée, Stress)
- **Paramètres individuels** : Fenêtre de réception, Corruption, Perte
- **Résumé** : Affichage des statistiques en temps réel (État, Paquets reçus, Cycles, Corruptions, Pertes)

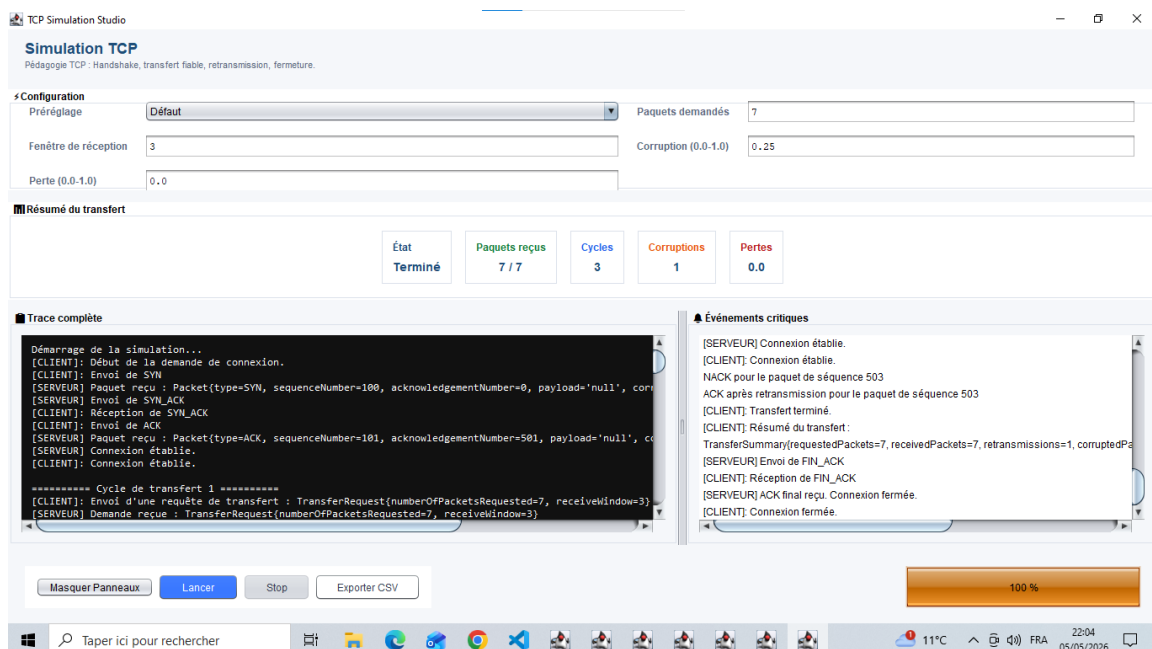


FIGURE 7.2 – Panneau de configuration avec préréglages et statistiques

7.4.3 Capture 3 : Résumé du Transfert

La figure 7.3 présente le panneau de résumé affichant les 5 métriques clés :

- **État** : TERMINÉ (connexion fermée avec succès)
- **Paquets reçus** : 7/7 (tous reçus)
- **Cycles** : nombre de cycles d'envoi nécessaires
- **Corruptions** : nombre de corruptions détectées et retransmises
- **Pertes** : nombre de paquets perdus (0 dans cet exemple)



FIGURE 7.3 – Panneau de résumé du transfert avec 5 métriques

7.4.4 Capture 4 : Sélection des Préréglages

La figure 7.4 montre le menu déroulant des 4 préréglages disponibles :

TABLE 7.2 – Préréglages de simulation disponibles

Préréglage	Paquets	Fenêtre	Corruption	Perte
Défaut	7	3	0.25	0.0
Rapide	5	2	0.15	0.0
Fiabilité élevée	8	3	0.05	0.0
Stress	12	2	0.35	0.1

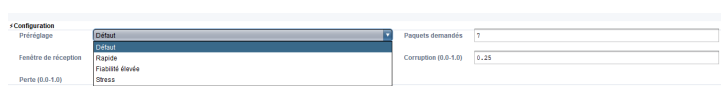


FIGURE 7.4 – Menu déroulant des 4 préréglages de simulation

7.4.5 Capture 5 : Export CSV

La figure 7.5 montre le fichier `transfer_summary.csv` généré après chaque simulation, contenant :

- En-têtes des colonnes : requestedPackets, receivedPackets, retransmissions, corruptedPacketsDetected, cycles, receiveWindow, completed
- Une ligne de données pour chaque simulation exécutée
- Format CSV standard compatible avec Excel et outils d'analyse

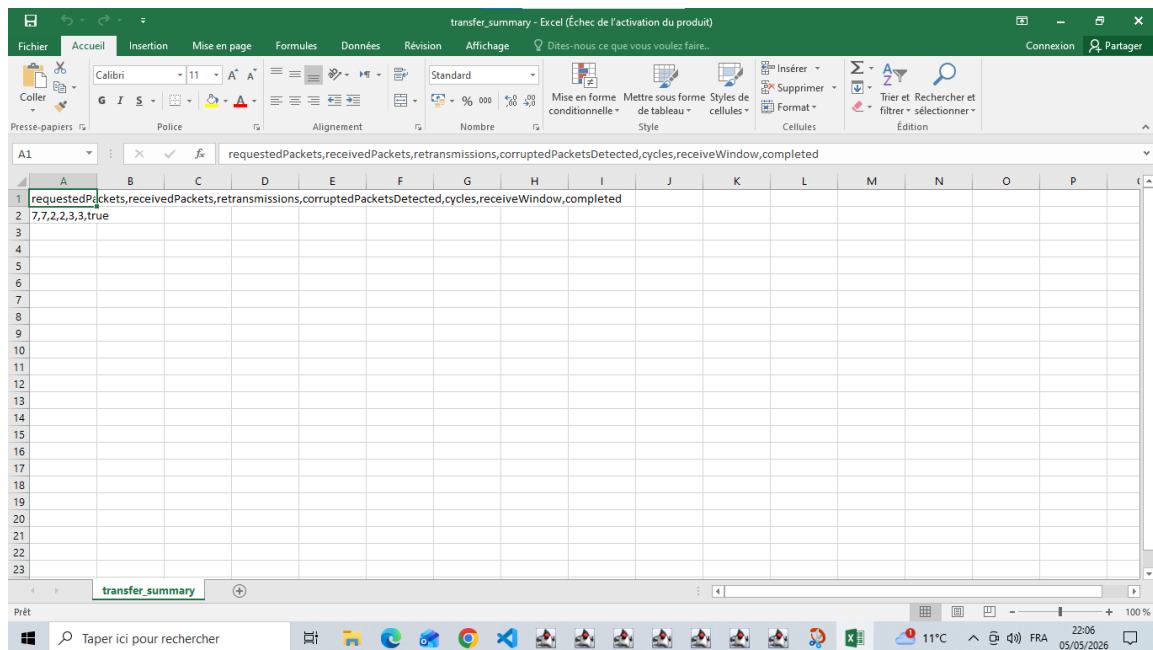


FIGURE 7.5 – Fichier d'export CSV avec résultats de simulation

Résumé visuel : Ces 5 captures montrent l'interface complète en action :

1. Console détaillée avec événements critiques filtrés
2. Configuration flexible avec préréglages (4 scénarios)
3. Résumé en temps réel (5 métriques clés)
4. Export automatique en CSV pour analyse

L'interface démontre la **pédagogie visuelle** du protocole TCP, permettant de comprendre facilement chaque étape de la simulation.

Annexe A

Configuration

A.1 config.properties

```
1 corruptionProbability=0.25
2 lossProbability=0.0
3 defaultPackets=7
4 defaultWindow=3
5 exportPath=transfer_summary.csv
```

Annexe B

Commandes d'Exécution

```
1  # Compilation
2  mvn clean compile
3
4  # Lancer GUI
5  java -cp target/classes fr.uvsq.tcpsim.gui.TcpGui
6
7  # Lancer CLI
8  java -cp target/classes fr.uvsq.tcpsim.cli.InteractiveCLI
9
10 # Lancer avec param tres
11 java -cp target/classes fr.uvsq.tcpsim.Main 10 5
12
13 # Tests
14 mvn test
15
16 # Build complet
17 mvn clean package
```

Bibliographie

- [1] POSTEL, J. (1981). *Transmission Control Protocol*. RFC 793, IETF.
- [2] BRADEN, R. (1989). *Requirements for Internet Hosts*. RFC 1122, IETF.
- [3] KUROSE, J. F., & ROSS, K. W. (2020). *Computer Networking* (8e éd.). Pearson.
- [4] STEVENS, W. R., & WRIGHT, G. R. (1994). *TCP/IP Illustrated : The Protocols*. Addison-Wesley.